

基于深度学习的机械臂自动分拣

1 任务描述

本任务使用python 深度学习程序识别物料类别，联合 Dobot vision 程序进行数据的 发送与接收。流程图如下：



图5-1 联合python与Dobot vision的数据流向

- Dobot vision 创建TCP 服务端，流程0发送数据，流程1 接收数据；
- Python IDE 创建客户端，先接收流程0发送的数据，采用深度模型使用预训练模型（或其他）读取 D\VisionMaster 采集图片，将识别结果发送给流程1
- 返回信号为 A/B/C/D 物料A：水果、B：蔬菜、C：服装、D：零食。当预测结果的概率最大值小于 80%，则判定为未知物料，发送 “error” 。

2. 任务实施

2.1 Dobot vision采集发送数据端

步骤1：将字符物料放置在传送带末端，打开视觉软件DobotVisionStudio，新建“通用方案”。在工具箱中，将“采集”子工具箱中的“相机图像”拖拽到流程编辑区。双击“相机图像”，对其参数进行设置，包括选择相机、触发设置等，单击执行获取字符物料图像。

步骤2：设置相机的颜色转换使相机的RGB图像转换为灰度图；



步骤3：在定位模块拖拽“快速特征匹配”与颜色转换模块相连，具体属性设置参见快速特征匹配模块，步骤3：将“定位”子工具箱中“快速匹配”拖拽到流程编辑区，并与“相机图像”连接。双击“快速匹配”设置基本参数，输入源选择“0相机图像1.图像数据”，ROI区域栏ROI创建选择矩形绘制，形状选择第二个，按住鼠标左键在图像中框选出ROI识别区域并调整矩形框的角度。

注：(ROI(region of interest), 感兴趣区域。机器视觉、图像处理中，从被处理的图像以方框、圆、椭圆、不规则多边形等方式勾勒出需要处理的区域，称为感兴趣区域，ROI)。



步骤4：设置放射变换，并将仿射变换与快速特征匹配相连。



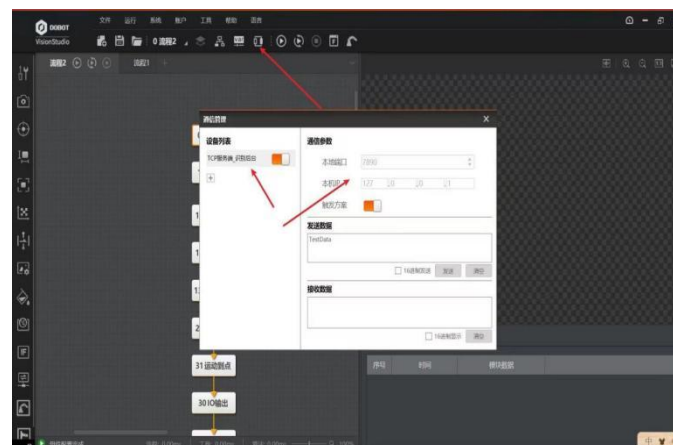
步骤5：设置图像存储位置，在这里设置得位置是 D:visionMaster.



步骤6：设置分支模块，判断是否正确



步骤7：设置发送数据模块。数据发送模块加载tcp 服务端，与后台建立通讯。注意此处得IP地址以及本地端口设置为 127.0.0.1,端口号为7890。也可以设置为其他端口号，但需要保持和python中建立得客户端端口号保持一致。发送数据“OK”给python 程序端，告知python程序端图像数据采集与处理已完成。





经过上述7个步骤完成流程0视觉采集的设置，相机将拍摄到的图片存储到D盘，使用数据发送模块建立tcp服务器端，发送图像采集完成信号至python程序。

2.1 Python 客户端深度识别物料类型

Python程序开发端创建tcp客户端，负责接收图像数据，使用与训练模型在线推理所采集得图片，获得识别结果后，将检测类型返回至dobot vision。若检测结果不存在，则返回为 error。

加载Model文件夹中的 Model-20230319.pkl 预训练模型，使用 \RobotRecognition\Main.py 运行在线推理，并返还类别结果。

- 加载库文件。

```
1. import os
2. import cv2
3. import numpy as np
4. from PIL import ImageFont, ImageDraw, Image
5. import torch
6. import torchvision
7. from torchvision import datasets, models, transforms
8. import torch.nn.functional as F
9. import torch.nn as nn
10. import time
11. import socket
```

- 创建TCP客户端，服务端IP为127.0.0.1，端口为7890。

```
12. # 创建一个TCP客户端
13. socket_client = socket.socket()
14. host = "127.0.0.1"
15. port = 7890
16. socket_client.connect((host,port))
```

- 模型推理。

```
17. # 模型推理
18. def modelpre(src_roi):
19.     grey_img = cv2.cvtColor(src_roi, cv2.COLOR_BGR2GRAY)
20.     tsfrm = transforms.Compose([
21.         transforms.Grayscale(3),
22.         transforms.Resize((224, 224)),
23.         transforms.ToTensor(),
```

```

24.     transforms.Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5])
25. ]
26. # 0.水果 1.蔬菜 2.服装 3.零食
27. classes = ('水果','蔬菜','服装','零食')
28. device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
29. model_eval = models.resnet18(pretrained=False)
30. num_ftrs = model_eval.fc.in_features
31. model_eval.fc = nn.Linear(num_ftrs, 4)
32. model_eval.load_state_dict(torch.load('./model/Model-202204.pkl', map_location=device))
33. # 在推理前, 务必调用model.eval()去设置dropout和batch normalization
    层为评估模式
34. model_eval.eval()
35. # OpenCV转PIL格式
36. image = Image.fromarray(cv2.cvtColor(grey_img, cv2.COLOR_GRAY2RGB))
37. # PIL图像数据转换为tensor数据, 并归一化
38. img = tsfrm(image)
39. # 图像增加1维[batch_size,通道,高,宽]
40. img = img.unsqueeze(0)
41. # 输出推理结果
42. output = model_eval(img)
43. # prob是4个分类的概率
44. prob = F.softmax(output, dim=1)
45. print(prob)
46. value, predicted = torch.max(output.data, 1)
47. label = predicted.numpy()[0]
48. # print(label)
49. pred_class = classes[predicted.item()]
50. # print(pred_class)
51. return label,pred_class

```

● 图像预处理。

```

52. # 图像预处理
53. def image_pre(image_source):
54.     # 图像处理: 灰度化
55.     dst = cv2.cvtColor(image_source,cv2.COLOR_RGB2GRAY)
56.     # 图像处理: 二值化 (可适当修改阈值范围)
57.     ret,dst_2=cv2.threshold(dst,100,230,cv2.THRESH_BINARY)
58.     # 图像处理: 形态学开运算
59.     kernel = np.ones((3,3),np.uint8)
60.     opening = cv2.morphologyEx(dst_2, cv2.MORPH_OPEN, kernel)
61.     # 图像处理: 包络轮廓
62.     contours, binary = cv2.findContours(opening,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)
63.     # 如果找不到轮廓, 则返回原图, 防止空载报错
64.     src_roi = image_source
65.     for i in range(0,len(contours)):
66.         # 打印轮廓面积
67.         area = cv2.contourArea(contours[i])
68.         # print(area)
69.         # 如果提取的轮廓面积少于50000, 跳过, 继续寻找轮廓
70.         if area < 50000:
71.             continue
72.         # 左上角坐标(x_l,y_l)、宽、高

```

```

73.         x_l, y_l, w, h = cv2.boundingRect(contours[i])
74.         # 中心点坐标
75.         x = (x_l+h)/2;
76.         y = (y_l+w)/2;
77.         # 获取所有轮廓（Blob连通区）最小外接矩形的（中心(x,y)，（宽,高
           ), 旋转角度）数据对象rect
78.         rect = cv2.minAreaRect(contours[i])
79.         # 获取最小外接矩形的4个顶点坐标(左上角、左下角、右下角、右上角)
80.         box = cv2.boxPoints(rect)
81.         # 数据类型转换
82.         box = np.int64(box)
83.         # 根据最小外接矩形的中心坐标与角度，构建一个旋转矩阵rot_img
84.         # 输入目标轮廓矩形中心点坐标center = rect[0]，矩形角度
           angle = rect[2]
85.         rot_img = cv2.getRotationMatrix2D(rect[0], rect[2], 1.0)
86.         # 使用前面获得的四个矩形顶点坐标数组[box]，在原图层绘制轮廓
87.         cv2.drawContours(image_source, [box], 0, (0, 255, 0), 1)

88.         # 利用旋转矩阵rot_img,原图层img实现中心仿射变换，变换后的图层尺
           寸保持不变
89.         # height = image_source.shape[0],
           width = image_source.shape[1]
90.         img_waf = cv2.warpAffine(image_source, rot_img, (image_sour
           ce.shape[0],image_source.shape[1]))
91.         # 仿射变换后得到img_waf图层中心点坐标和目标轮廓最小外接矩形中心
           点一致，
92.         # 根据数据对象rect提取出矩形中心点坐标和宽高x, y, w, h，分别得到
           矩形行(x)和列(y)的起始点和结束点
93.         # 在img_waf中剪裁出纠正角度后的目标图层src_roi
94.         # 语句原型(img_waf[y-int(h/2): y+int(h/2)+4, x-
           int(w/2): x+int(w/2)+4])
95.         src_roi = img_waf[int(rect[0][1])-
           int((rect[1][1])/2)+4:int(rect[0][1]+int((rect[1][1])/2)-4,
           int(rect[0][0])-
           int((rect[1][0])/2)+4:int(rect[0][0]+int((rect[1][0])/2)-4]
96.         cv2.imwrite('./image/img.jpg', src_roi)
97.         break
98.     return src_roi
99.     ● 显示推理结果。

100. # 显示推理结果
101. def showResult(img,text):
102.     # 设置图片显示分辨率
103.     newimg = cv2.resize(img,(640,480))
104.     fontpath = "font/simsun.ttc"
105.     font = ImageFont.truetype(fontpath, 48)
106.     img_pil = Image.fromarray(newimg)
107.     draw = ImageDraw.Draw(img_pil)
108.     # 绘制文字信息
109.     draw.text((10, 100), text, font=font, fill=(0, 0, 255))
110.     bk_img = np.array(img_pil)
111.     # 获取实时时间作为命名，将识别图片和结果保存到文件夹
112.     str_time = time.strftime("%Y%m%d%H%M%S", time.localtime(time.t
           ime()))
113.     img_name = str_time + '.jpg'
114.     cv2.imwrite('./result/' + img_name, bk_img)
115.     cv2.imshow('frame', bk_img)

```

```

116.     # 持续显示3s
117.     key = cv2.waitKey(3000)
118.     cv2.destroyAllWindows()
    ● ROI区域裁剪函数。
119. # ROI区域裁剪函数
120. def image_roi():
121.     image_source = cv2.imread('D:/VisionMaster/image.jpg')
122.     if image_source is None:
123.         print('Image is not found.')
124.         return ''
125.     # 裁剪坐标为[y0:y1, x0:x1]
126.     # ROI区域(用户根据需求修改)
127.     img_roi = image_source[445:1373, 877:1595]
128.     # 保存图像
129.     cv2.imwrite('./image/roi.jpg',img_roi)
130.     return img_roi
    ● 清空缓存图片。
131. # 清空缓存图片
132. def remove_result():
133.     path = './result/'
134.     # 判断是否存在result文件夹
135.     if os.path.exists(path):
136.         for i in os.listdir(path):
137.             # 遍历拼接文件路径
138.             path_file = os.path.join(path, i)
139.             # 判断该路径对象是否为文件
140.             if os.path.isfile(path_file):
141.                 # 删掉文件
142.                 os.remove(path_file)
143.     else:
144.         # 新建一个result文件夹
145.         os.mkdir(path)
    ● 主程序入口。
146. # 主程序入口
147. if __name__ == "__main__":
148.     # 每次运行前清空上次运行识别结果缓存图片
149.     remove_result()
150.     print('Waiting...')
151.     while True:
152.         data = socket_client.recv(1024).decode('utf-8')# TCP读取数据
153.         print(data)
154.         msg = ''
155.         if data == 'ok':
156.             # 方案一：原图进行ROI裁剪
157.             # img_roi = image_roi()
158.             # # 将裁剪后的图像进行预处理
159.             # src_roi = image_pre(img_roi)
160.             # 方案二 仿射变换方案
161.             src_roi = cv2.imread('D:/VisionMaster/image.jpg')
162.             # 将图像预处理后的图像进行模型推理
163.             label, pred_class = modelpre(src_roi)
164.             # 打印推理结果
165.             print(label, pred_class)

```

```

166.         # 显示推理结果
167.         showResult(src_roi, pred_class)
168.         if label == 0:
169.             msg = 'A'
170.         elif label == 1:
171.             msg = 'B'
172.         elif label == 2:
173.             msg = 'C'
174.         elif label == 3:
175.             msg = 'D'
176.         socket_client.send(msg.encode())
177.     else:
178.         msg = 'null'
179.         socket_client.send(msg.encode())
180.         print('data error!')

```

(3) 运行结果

单击python运行Main.py 程序Python Shell窗口出现“waiting...”

Dobot vision 执行流程0 给图像识别后台发送ok数据，Main.py脚本接收“ok”；

Main.py 然后将识别结果打印在图片上，如识别结果为“零食”，同时在Python Shell窗口也将识别结果打印出来,并将结果结果”D“发送给Dobot Vision 流程2



Main程序运行结果

由于实验室电脑联网等原因，同学们可在自己电脑端完成深度学习的配置，验证深度学习识别结果，使用调试助手手法数据验证调试助手与python 程序的数据通讯。将识别结果手动输入至 实验室台式机一下程序：手动输入msg的实验结果 ‘A/B/C/D/ERROR’

1-运行333.py文件，创建客户端，等待接收数据

2-运行流程0，创建服务端，发送数据至333.py所创建的客户端

3-333.py接收到数据data为ok，打印结果，赋值msg为物料类型A（也可为其他类型），发送数据msg

4-流程1创建服务端，接收由333.py发送的数据

下为创建的333.py脚本文件

```

import os
import socket
socket_client = socket.socket()
host = "127.0.0.1"
port = 7890
socket_client.connect((host,port))
if __name__ == "__main__":
    print('Waiting...')
    while True:
        data = socket_client.recv(1024).decode('utf-8')
        print(data)
        msg = ''

```

```

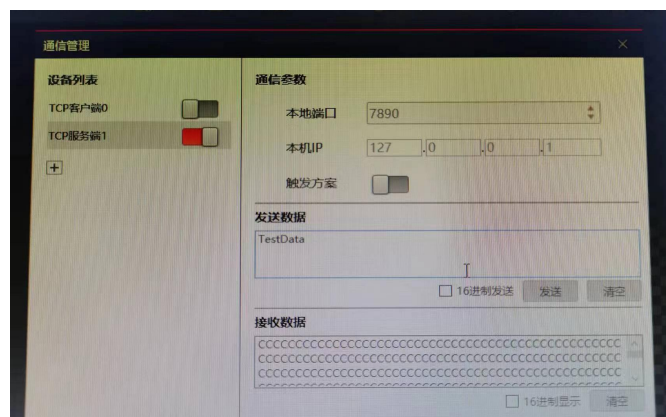
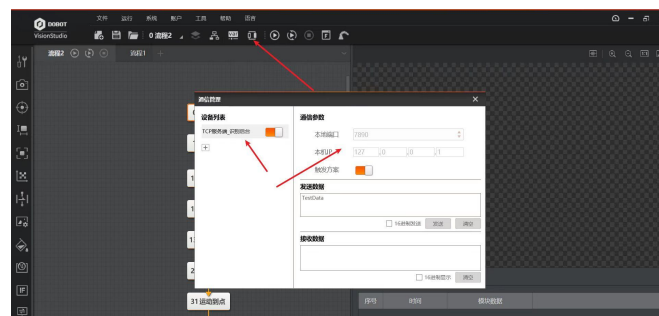
if data == 'ok':
    msg = 'A'
    socket_client.send(msg.encode())

```

2.3 Dobot Vision 接收数据与执行设置

由流程 1 完成数据接收物料分类并分别放置到物料盘，下面进行流程 1 的设置。

步骤 1：创建 TCP 服务器后台，设置本地端口，单次执行程序，观察框内的接收数据显示，用于接受 333.py 发送的数据。



步骤 2：设置字符比较模块，确定发送过来的字符跟设定一致。



步骤 3：设置条件检测模块，当条件满足时触发下面的输出



步骤 4：设置分支模块



步骤 5：设置机械臂方向模块。



步骤 6：为保证安全，机械臂 XY 平面的移动和 Z 方向的运动不宜同时进行，通过调试机械臂设置两个运动到点模块（具体调试方法见验证标定精度步骤 9）



步骤 7：设置 IO 输出模块，IO 输出模块控制机械臂的吸盘。



步骤 8：设置两个运动到点模块。





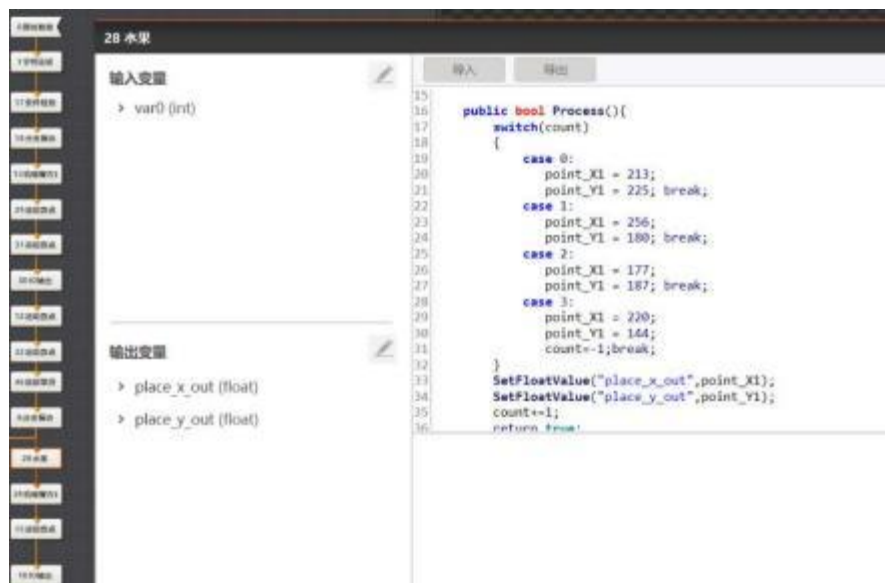
步骤 9：如图 设置一个延时模块。



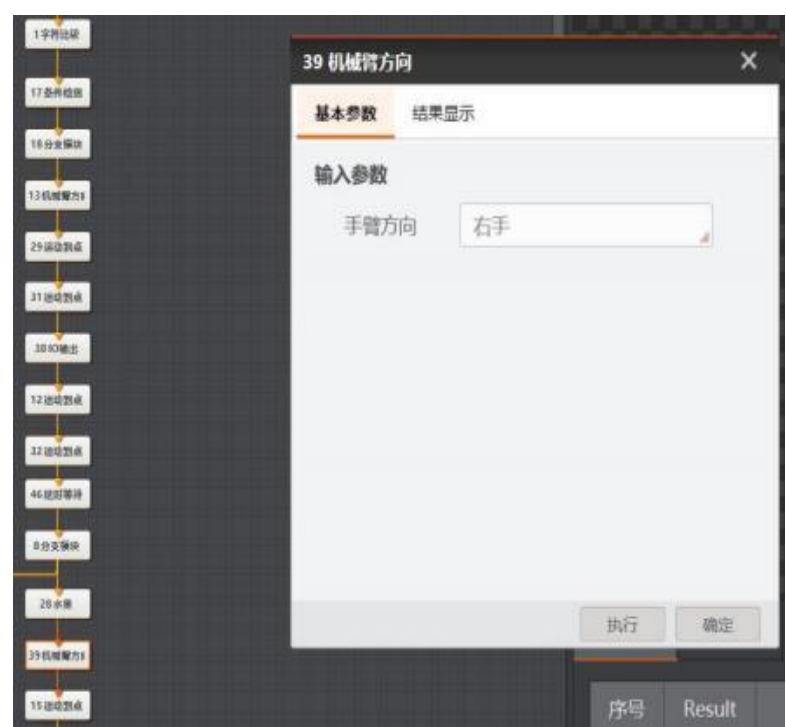
步骤 10：设置一个分支模块，根据物体不同类别执行不同分支的命令



步骤 11：设置脚本模块，确定每一类别的目标坐标值，这里的 X，Y 值应根据不同目标更改相应设置，



步骤 12：如图 设置机械臂方向。



步骤 13：如图 设置运动到点模块。



步骤 14：设置 IO 输出模块。



步骤 15：设置运动到点模块。



步骤 16：两个流程图完成了抓取物料，识别物料，物料分类，放置物料等任务

